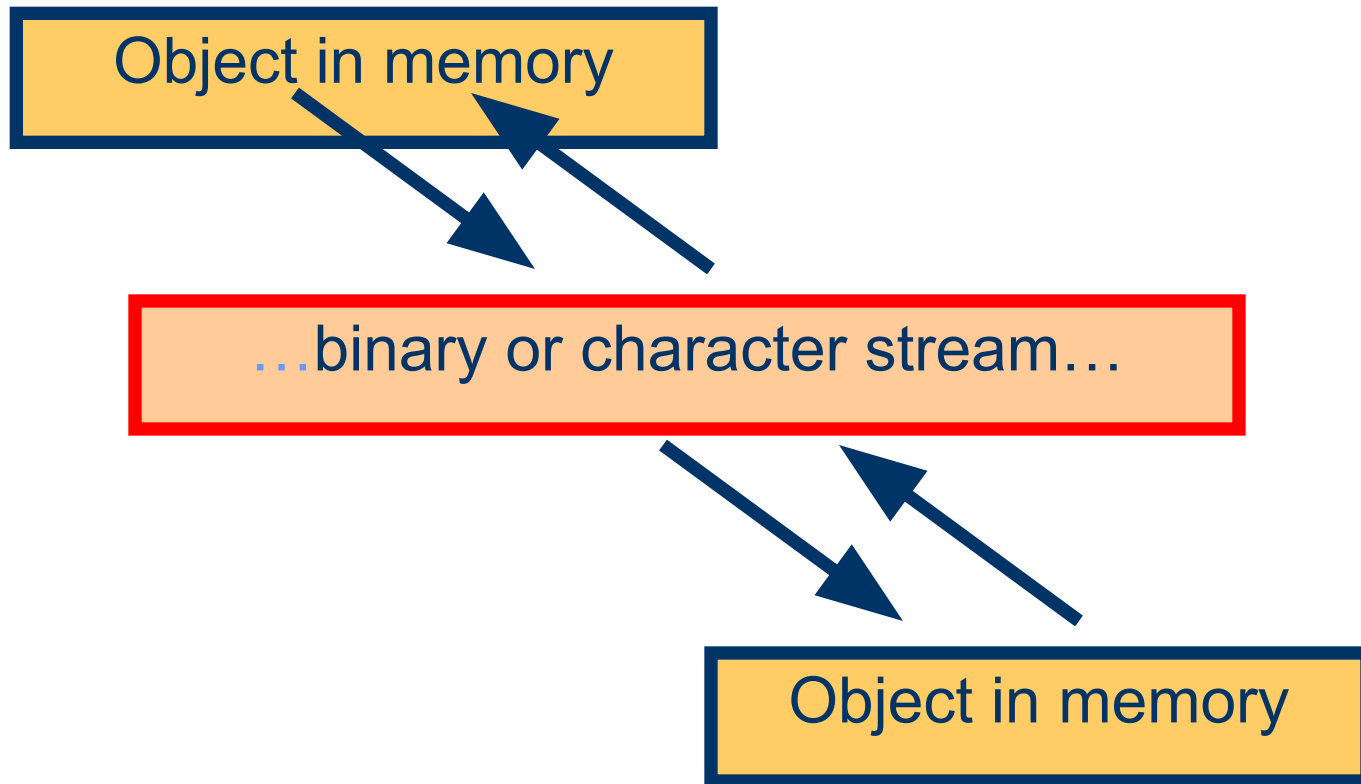


Serialization

- ◆ What is Serialization?
- ◆ System.Serialization
- ◆ Scenarios in Serialization
- ◆ Basic Serialization
- ◆ Custom Serialization

Serialization/Deserialization



Serialization

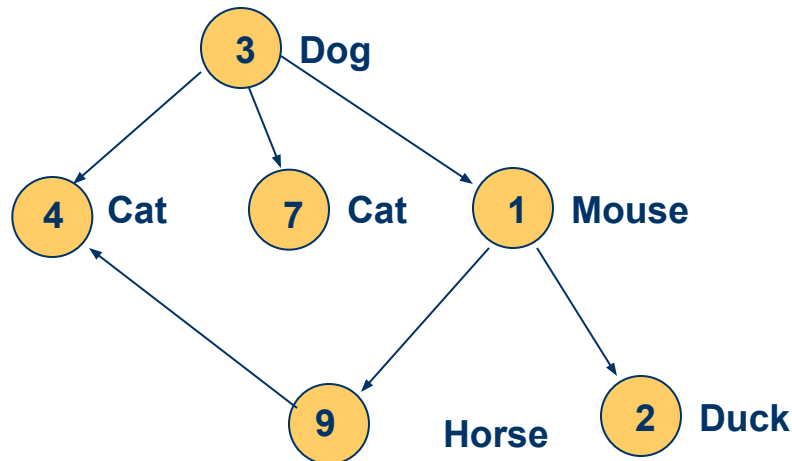
What is Serialization

- ◆ Serialization is the process of converting an object, or a connected graph of objects, stored within computer memory, into a linear sequence of bytes
- ◆ Use the sequence of bytes in several ways:
 - Send it to another process
 - Send it to the clipboard, to be browsed or used by another application
 - Send it to another machine
 - Send it to a file on disk

Serialization

Object Graph

- ◆ What is an object graph?
 - An object graph is a set of objects with some set of references to each other
 - The most obvious problem is how to represent the links between the objects in the Serialized stream



Serialization

How Serialization Works

- ◆ Because run-time metadata 'knows' about each object's layout in memory, and its field and property definitions, you can serialize objects automatically, without having to write code to serialize each field
- ◆ The serialized stream might be encoded using XML, or a compact binary representation
- ◆ The format is decided by the the Formatter object that you call:
 - Binary
 - SOAP
 - Custom

Serialization

FileStream Example

```
class SerializeExample{

public static void Main(String[] args)
{
    ArrayList l = new ArrayList();
    for (int x=0; x< 100; x++) {
        l.Add (x);
    } // create the object graph
    FileStream s = File.Create("data.dat");
    // create the filestream
    BinaryFormatter b = new BinaryFormatter();
    // create the BinaryFormatter
    b.Serialize(s, l);
    // serialize the graph to the stream
} // end main
} // end class
```

Serializaiton Deserialize Example

```
using System; using System.IO;
using System.Collections;
using System.Serialization;
using System.Serialization.Formatters.Binary;

class DeSerialize
{
    public static void Main(String[] args) {
        FileStream s = File.Open("data.dat");
        // open the filestream
        BinaryFormatter b = new BinaryFormatter();
        // create the formatter
        ArrayList p = (ArrayList) b.Deserialize(s);
        // deserialize
        p.ToString(); // print out the new object graph
    } // end Main
} // end Class DeSerialize
```

Serialization

Basic Serialization

- ◆ A Type is NOT Serializable unless Type is specifically marked as Serializable
- ◆ The Serializable Attribute

```
[Serializable] public class MyClass {}
```

- ◆ The Non-Serializable Attribute

```
[Serializable] public class MyClass {  
    [NotSerialized] int _cashSize;  
}
```


.NET Serialization Facilities

Take an extremely simple C# class:

```
public class InitialConfiguration
{
    public enum Difficulty {hard, medium, easy};
    public InitialConfiguration() { }
    public Difficulty starting = Difficulty.medium;
}
```

.NET Serialization Facilities

Use .NET library functions to serialize it:

```
InitialConfiguration conf = new InitialConfiguration();
```

```
XmlSerializer ser  
    = new XmlSerializer(typeof(InitialConfiguration));
```

```
XmlTextWriter writer = new XmlTextWriter(  
    stream, System.Text.Encoding.UTF8);
```

```
ser.Serialize(writer, conf);
```

.NET Serialization Facilities

Get XML:

```
<?xml version="1.0" encoding="utf-8"?>
<InitialConfiguration
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <Starting>medium</starting>
</InitialConfiguration>
```

Serializaiton

XMLSerializaiton Example

```
class SerializeExample{
public static void Main(String[] args)
{
    MySerializableClass myObject = new MySerializableClass();

    // Insert code to set properties and fields of the
    object. XmlSerializer mySerializer = new
    XmlSerializer(typeof(MySerializableClass));

    // To write to a file, create a StreamWriter object.
    StreamWriter myWriter = new
    StreamWriter("myFileName.xml");

    mySerializer.Serialize(myWriter, myObject);
    myWriter.Close();
}
}
```

Serializaiton

XMLSerializaiton Example

```
// Call the Deserialize method and cast to the object
//type.
myObject = (MySerializableClass)
mySerializer.Deserialize(myFileStream)
} // end main

} // end class
```

Serializaiton Deserialize Example

```
class DeSerialize
{
    MySerializableClass myObject;
    // Construct an instance of the XmlSerializer with the //
    type of object that is being deserialized.
    XmlSerializer mySerializer = new
    XmlSerializer(typeof(MySerializableClass));

    // To read the file, create a FileStream.
    FileStream myFileStream = new
    FileStream("myFileName.xml", FileMode.Open);

    // To write to a file, create a StreamWriter object.
    StreamWriter myWriter = new
    StreamWriter("myFileName.xml");
    mySerializer.Serialize(myWriter, myObject);
    myWriter.Close();
} // end Main
} // end Class DeSerialize
```

Serialization

Customize Serialization

- ◆ Implementing ISerializable interface
- ◆ IDeserializationEventListener
- ◆ Custom Formatters

Serialization

ISerializable Interface

- ◆ Customize the serialization process
- ◆ If a class implements ISerializable, that interface will always be called in preference to default serialization.
- ◆ The ISerializable interface is only contains one method:

```
void GetObjectData (SerializationInfo info,  
    StreamingContext context); And an implied  
constructor that may be private. private <TypeName>  
(SerializationInfo info, StreamingContext)
```


Serialization

IDeserializationEventListener

- ◆ If an object implements `IDeserializationEventListener`, the serialization infrastructure will call that class' `OnDeserialization` method as soon as the entire graph has been deserialized and all fix-ups completed
- ◆ Provide a reasonable opportunity for objects that need to do fix-ups based on the state of their children

Serialization

Custom Formatter

- ◆ Implementing IFormatter Interface:

```
public interface IFormatter:
{ //Properties
    SerializationBinder Binder { get; set; }
    StreamingContext Context { get; set; }
    ISurrogateSelector SurrogateSelector { get; set; }
    //Methods
    object Deserialize(Stream serializationStream);
    void Serialize(Stream serializationStream, object
graph);
}
```

Conclusion

- ◆ Types' metadata can be explored with Reflection
- ◆ Reflection provides dynamic type system
- ◆ The Federated Services Model is one of the core concepts for designing .NET applications in Internet
- ◆ Key .NET Remoting scenarios are:
 - Web Services Anywhere
 - CLR Object Remoting
- ◆ Serialization is the process of converting an object, or a connected graph of objects, stored within computer memory, into a linear sequence of bytes